

Cut Through the Noise. See What's Real.

Project Name	HyperFilter.ai — AI Noise & Bias Filter
Type	Chrome Browser Extension + AI Backend
Target	Resume / LinkedIn Portfolio Project
Build Time	3–4 Weeks (part-time alongside college)
Tech Stack	JavaScript, Chrome Extensions API, Node.js, LLM API (Claude/OpenAI)
Difficulty	Intermediate → Advanced (perfect for pre-final year)
GitHub Visibility	Public — showcase-ready

■ What HyperFilter.ai Does

HyperFilter.ai is a Chrome browser extension that reads text on any webpage and uses AI to detect:

- Corporate buzzwords & empty jargon ("synergy", "disruptive", "circle back")
- Logical fallacies (strawman, ad hominem, slippery slope)
- Cognitive biases (confirmation bias, appeal to authority, FOMO manipulation)
- Fake urgency & emotional manipulation tactics
- Misinformation red flags (unsourced claims, exaggerated stats)

Results are highlighted directly on the page with color-coded underlines. Hovering any highlight shows a beautiful tooltip explaining what was detected and why.

■ Unique Features (Your Differentiators)

Feature 1 — Live Verdict Score

Every page gets an overall score: "82% Substance | 18% Noise". Shown as a floating badge. Instantly shareable for LinkedIn demos.

Feature 2 — Bias Heatmap Mode

Toggle a full-page heatmap overlay where paragraphs are colored red→green based on bias density. Like a grammar checker, but for cognitive manipulation.

Feature 3 — Platform Personality Modes

Auto-detects the platform: LinkedIn → Corporate Jargon mode, Twitter → Rage Bait mode, Reddit → Echo Chamber mode, News → Sensationalism mode.

Feature 4 — Roast Mode ■

Optional sarcastic takedowns: "Verdict: This post is 94% buzzwords. The author discovered how to say nothing in 300 words." — your LinkedIn viral clip.

Feature 5 — Scan History Dashboard

Popup dashboard showing all pages scanned this week, their scores, and which sites are most biased. Beautiful charts for resume screenshots.

Feature 6 — Export Report

One-click export of full analysis as JSON. Great for showing product thinking in interviews.

■■ System Architecture

■ LAYER 1	■ LAYER 2	■ LAYER 3
Chrome Extension Frontend Content Scripts + Popup UI + Background Worker	Node.js Backend + LLM API Express server + Prompt Engineering + JSON response	Chrome Storage API Scan history, user preferences, cached results

Data Flow

```
User visits a webpage

↓

Content Script reads page text (DOM scraping)

↓

Background Service Worker receives the text

↓

Sends POST request → Node.js Backend (localhost:3000)

↓
```

Backend chunks text → sends to LLM API with engineered prompt

↓

LLM returns JSON: { offending_phrases[], verdict_score, summary }

↓

Content Script injects highlights + tooltips into the DOM

↓

Results saved to Chrome Storage API for History Dashboard

■ Project File Structure

hyperfilter/

■ ■ ■ extension/ ← Chrome Extension (Frontend)

■ ■ ■ ■ manifest.json ← Extension config & permissions

■ ■ ■ ■ content/

■ ■ ■ ■ content.js ← DOM scraping + highlight injection

■ ■ ■ ■ tooltip.css ← Tooltip & highlight styles

■ ■ ■ ■ background/

■ ■ ■ ■ service-worker.js ← API calls + message routing

■ ■ ■ ■ popup/

■ ■ ■ ■ popup.html ← Extension popup UI

■ ■ ■ ■ popup.js ← Popup logic + scan trigger

■ ■ ■ ■ popup.css

■ ■ ■ ■ dashboard/

■ ■ ■ ■ dashboard.html ← Scan history page

- ■ ■■ dashboard.js ← Charts + history rendering
- ■ ■■ dashboard.css
- ■■ icons/ ← Extension icons (16, 48, 128px)
-
- backend/ ← Node.js Backend (Brain)
- ■■ index.js ← Express server entry point
- ■■ routes/analyze.js ← /analyze POST endpoint
- ■■ services/
- ■ ■■ llm.js ← LLM API integration
- ■ ■■ chunker.js ← Text chunking logic
- ■ ■■ promptBuilder.js ← Prompt engineering
- ■■ .env ← API keys (NEVER commit this!)
- ■■ package.json
-
- README.md ← GitHub showcase file
- demo/ ← Screenshots + GIFs

■■ Tech Stack

Chrome Extensions API	MV3 (Manifest V3) — latest standard, impresses interviewers
JavaScript	TypeScript preferred — shows maturity. JS is fine to start.
Content Scripts	For reading & manipulating live webpage DOM
Service Workers	Background script for API calls (MV3 requirement)
Node.js + Express	Lightweight backend to proxy LLM API calls
LLM API	Claude API (Anthropic) — best for nuanced text analysis

Chrome Storage API	Built-in browser storage for scan history
Chart.js	For the history dashboard charts
dotenv	Environment variable management for API keys

■ The Core LLM Prompt (Your Secret Weapon)

This is what you send to the AI. Well-engineered prompts are a real skill — mention this in interviews.

```
You are HyperFilter.ai, an expert cognitive bias and misinformation detector.
```

```
Analyze the following text and identify:
```

1. Corporate buzzwords / empty jargon
2. Logical fallacies (name the specific fallacy)
3. Cognitive bias triggers (FOMO, appeal to authority, etc.)
4. Emotional manipulation tactics
5. Unsourced or exaggerated claims

```
Return ONLY a valid JSON object in this exact format:
```

```
{  
  
  "verdict_score": "<0-100, where 100 = pure substance, 0 = pure BS>,"  
  
  "platform_detected": "<linkedin|twitter|reddit|news|other>,"  
  
  "summary": "<one witty sentence verdict>,"  
  
  "roast": "<sarcastic one-liner for Roast Mode>,"  
  
  "offending_phrases": [  
  
    {  
  
      "phrase": "<exact text from the input>,"  
  
      "type": "<buzzword|fallacy|manipulation|bias|claim>,"
```

```

"severity": "<low|medium|high>",

"explanation": "<why this is problematic>"

}

]

}

```

Text to analyze:

```

"""

```

```

{TEXT_CHUNK}

```

```

"""

```

■ ■ Build Stages & Checklist

Complete each stage, tick the boxes, then send the updated file to continue building with AI assistance.

STAGE 1 — Project Setup & Basic Extension ■ 3–4 days

- | | |
|---|---|
| 1 | Create project folder structure as shown above |
| 2 | Write manifest.json with required permissions (activeTab, storage, scripting) |
| 3 | Create a basic content.js that logs page text to console |
| 4 | Create popup.html with a Scan button (no functionality yet) |
| 5 | Load extension in Chrome (chrome://extensions → Developer Mode → Load Unpacked) |
| 6 | Verify extension loads without errors in Chrome |
| 7 | Initialize Git repo and push to GitHub |
| 8 | [CHECKPOINT] Extension loads in Chrome ✓ |

STAGE 2 — DOM Scraping & Text Extraction ■ 2–3 days

- | | |
|---|--|
| 1 | Write content.js to extract all readable text from the page |
| 2 | Filter out nav, footer, scripts, ads — only main content |
| 3 | Implement platform detection (linkedin.com, twitter.com, etc.) |

4	Send extracted text to background service-worker via chrome.runtime.sendMessage
5	Console.log the extracted text to verify quality
6	Handle edge cases: empty pages, paywalled content, SPAs
7	[CHECKPOINT] Clean text extracted from LinkedIn & Twitter ✓

STAGE 3 — Backend Setup & LLM Integration ■ 4–5 days

1	Initialize Node.js project in backend/ folder (npm init)
2	Install express, axios, dotenv, cors
3	Create /analyze POST endpoint in analyze.js
4	Add API key to .env file (add .env to .gitignore immediately!)
5	Implement text chunking in chunker.js (max ~2000 tokens per chunk)
6	Write the engineered prompt in promptBuilder.js
7	Call LLM API and parse the JSON response
8	Test backend with Postman or Thunder Client
9	Connect service-worker.js to call the backend
10	[CHECKPOINT] Backend returns valid JSON analysis ✓

STAGE 4 — Highlight Injection & Tooltip UI ■ 4–5 days

1	Write highlight injection logic in content.js
2	Match offending_phrases from JSON to text on the page
3	Wrap matched phrases in tags with color-coded classes
4	Design tooltip card in tooltip.css (severity colors: red/orange/yellow)
5	Show tooltip on hover with phrase type + explanation
6	Implement verdict score badge (floating corner element)
7	Test on LinkedIn, Twitter, and a news site
8	[CHECKPOINT] Highlights + tooltips working on live pages ✓

STAGE 5 — Popup UI + Roast Mode ■ 2–3 days

1	Design popup.html with verdict score display
2	Add Scan Page button that triggers content script

3	Add Roast Mode toggle (shows sarcastic summary)
4	Show platform detected + category breakdown in popup
5	Add loading spinner while API call is in progress
6	Style popup professionally (this is in your demo video!)
7	[CHECKPOINT] Popup shows full analysis results ✓

STAGE 6 — Scan History Dashboard ■ 3–4 days

1	Save each scan result to Chrome Storage API
2	Build dashboard.html as a full extension page
3	Display scan history in a table (URL, date, score)
4	Add Chart.js bar chart showing scores over time
5	Add Most Biased Sites leaderboard section
6	Add Export as JSON button
7	Link dashboard from popup via View History button
8	[CHECKPOINT] History dashboard shows scan data + charts ✓

STAGE 7 — Polish, README & LinkedIn Demo ■ 2–3 days

1	Write a killer README.md (problem → solution → tech → screenshots → install guide)
2	Add extension icons (use Figma or Canva for quick icon design)
3	Add screenshots and GIFs to README
4	Record 30-second demo video (Loom or OBS)
5	Test on 5 different websites and fix edge cases
6	Final GitHub push — check repo looks clean and professional
7	Write LinkedIn post with demo video attached
8	[CHECKPOINT] Live on GitHub + posted on LinkedIn ✓

■ Interview Talking Points

When an interviewer asks about this project, say:

→ "I built a Chrome Extension using Manifest V3 — the latest extension standard — which required learning Service Workers instead of background pages."

- "The core challenge was DOM manipulation at scale — extracting clean, readable text from any webpage without grabbing nav, ads, or boilerplate."
- "I engineered a structured JSON prompt that forces the LLM to return machine-parseable output rather than free text — that's the reliability trick."
- "Text chunking was an interesting problem — LLMs have context window limits, so I split text into ~2000 token chunks and merged the results."
- "Platform detection lets me switch prompt personality per site — LinkedIn mode is aggressive on buzzwords, news mode focuses on sourcing."

■ Resume Bullet Points (Copy-Paste Ready)

HyperFilter.ai — AI Cognitive Bias Detector | JavaScript, Chrome Extensions API, Node.js, LLM API

- Built a Chrome Extension (MV3) that detects cognitive biases, logical fallacies & corporate buzzwords in real-time on any webpage using a custom-engineered LLM prompt
- Implemented DOM scraping with text chunking (2000-token chunks) to handle context window limits while maintaining analysis accuracy across long-form content
- Designed 6 unique features: Live Verdict Score, Bias Heatmap, Platform Personality Modes, Roast Mode, Scan History Dashboard, and JSON Export
- Achieved platform-specific detection by auto-identifying LinkedIn, Twitter, Reddit & news sites and switching prompt persona accordingly
- Persisted scan history using Chrome Storage API with Chart.js visualizations

■ Your Progress Tracker

Stage 1 — Setup & Basic Extension	☐ Not Started
Stage 2 — DOM Scraping	☐ Not Started
Stage 3 — Backend & LLM Integration	☐ Not Started

Stage 4 — Highlight Injection & Tooltips	☐ Not Started
Stage 5 — Popup UI & Roast Mode	☐ Not Started
Stage 6 — Scan History Dashboard	☐ Not Started
Stage 7 — Polish, README & LinkedIn Demo	☐ Not Started

age and share it back to continue building with AI assistance.

You've got this. Build it. Ship it.